RatesTools on Cornell BioHPC

Installation, configuration, chromosome-scattered GATK4 calling, resource control, troubleshooting, and rerun strategy

Working analysis: Atlantic silverside trio (LMJM2, LMPF2, LMPJ001), chromosome-level reference, approximately 90x coverage per sample.

Pipeline base: campanam/RatesTools v1.2.5, commit `ce3358c7beebf8a7afd277990dc298d2daa1dcdc`.

Purpose: This record documents the exact operational decisions and source-code changes made so the workflow can be reproduced later without reconstructing the debugging process.

Quick-start checklist

• Run from the analysis directory, but point Nextflow to the locally modified ratestools.nf.

• Always use -resume and preserve the work/ directory.

• Use absolute paths for real inputs.

• Verify dam, sire, chr_file, RepeatMasker species, and contig thresholds manually.

• Use one HaplotypeCaller task per sample x chromosome and cap executor CPUs on shared hardware.

• Run inside screen so SSH disconnects do not terminate the workflow.

1. Directory layout and launch model

The analysis files and the pipeline source are kept separate. This follows the RatesTools recommendation to maintain a run-specific configuration directory while still allowing local source-code modifications.

```
/workdir/nina_july2026/silverside_progeny/
■■■ RatesTools/                    # Local clone; modified ratestools.nf
■■■ RatesTools_LMPJ001/            # Analysis launch directory
■   ■■■ nextflow.config
■   ■■■ run_ratestools.sh
■   ■■■ LMPJ001_fastq.csv
■   ■■■ hc_intervals.txt
■   ■■■ work/                      # Nextflow cache - preserve
■   ■■■ test_results/
■■■ genome/
    ■■■ Mmenidia_refgenome_anchored.all_renamed_v2_core.fasta
```

Running `nextflow run campanam/RatesTools` uses the Nextflow-cached GitHub project, not the locally edited clone. The modified workflow must therefore be launched by providing the full path to `ratestools.nf`.

Production launch command

```
cd /workdir/nina_july2026/silverside_progeny/RatesTools_LMPJ001

~/bin/nextflow-25.10.5 run   /workdir/nina_july2026/silverside_progeny/RatesTools/ratestools.nf   -c nextflow.config   -profile
```

Screen session

```
screen -S ratestools
./run_ratestools.sh

# Detach:
Ctrl-A, then D

# Reattach:
screen -d -r ratestools

# Confirm current shell is inside screen:
echo $STY
```

2. Environment setup

Nextflow itself requires Java 17 or newer. BioHPC provides Java 21 as a module. The workflow uses Conda/Mamba environments for most tools, while GenMap was configured from a working system path.

run_ratestools.sh

```
#!/bin/bash

module load java/21.0.1

export PATH=/programs/sambamba-0.7.1:$PATH
export PATH=/programs/ruby/bin:$PATH
export PATH=/programs/genmap/bin:$PATH

~/bin/nextflow-25.10.5 run   /workdir/nina_july2026/silverside_progeny/RatesTools/ratestools.nf   -c nextflow.config   -profile
```

A backslash used for shell line continuation must be the final character on the line. A trailing space after the backslash breaks the command.

3. Why configure.sh was not trusted

The provided configure.sh was useful as a starting point but did not produce a reliable final configuration on BioHPC.

• The script used macOS/BSD sed syntax (sed -i ''), which failed on Linux.

• Replacement commands broke when absolute paths contained forward slashes.

• Several entered answers did not appear in the resulting configuration.

• Some defaults remained silently unchanged, including parent names and contig thresholds.

• The script did not validate whether paths such as chr_file actually existed.

The effective configuration should therefore always be inspected manually:

```
~/bin/nextflow-25.10.5 config   /workdir/nina_july2026/silverside_progeny/RatesTools   -profile conda
```

Parameters that required manual verification

Parameter	Correct intent / action
refseq	Absolute path to the chromosome-level reference FASTA
libraries	Absolute path to LMPJ001_fastq.csv
readDir	Absolute path to the gzipped FASTQ directory
gatk_build	4
dam / sire	Exact parental sample IDs from CSV, e.g. LMJM2 and LMPF2
chr_file	Existing chromosome list or NULL
min_contig_length	100000 for the intended 100 kb threshold
min_filt_contig_length	100000 for the intended 100 kb threshold
rm_species	Must not be Felidae; choose an appropriate supported fish taxon or custom repeat library
hc_intervals	Absolute path to hc_intervals.txt

4. Input preparation

FASTQ compression

```
# Compress and remove originals:
gzip *.fq

# Or parallel compression:
pigz -p 8 *.fq
```

CSV filenames must exactly match actual files, including the .gz suffix. A useful validation is:

```
awk -F, 'NR>1 {print $3; print $4}' LMPJ001_fastq.csv |
while read f; do
  test -f "/fs/cbsubscb16/storage/silverside_progeny/batch_full/trimmed/$f"     || echo "MISSING: $f"
done
```

Chromosome interval file

```
cut -f1   /workdir/nina_july2026/silverside_progeny/genome/Mmenidia_refgenome_anchored.all_renamed_v2_core.fasta.fai   > /workd

wc -l hc_intervals.txt
cat hc_intervals.txt
```

For the core reference, this should contain one chromosome name per line (24 lines), with no header.

5. Core workflow modification: chromosome-scattered HaplotypeCaller

The original pipeline ran one whole-genome GATK4 HaplotypeCaller process per sample. In practice, each process used about one CPU core and required multiple days. The modification scatters calling by chromosome, yielding 3 samples x 24 chromosomes = 72 independent tasks.

Replacement callVariants process

```
process callVariants {

    // Run GATK HaplotypeCaller separately for each sample and chromosome

    label 'gatk'
    tag "${fix_bam.simpleName}:${interval}"

    input:
    tuple path(fix_bam), path(fix_bai), val(interval)
    path refseq
    path "*"

    output:
    tuple val(fix_bam.simpleName),
          val(interval),
          path("${fix_bam.simpleName}.${interval}.g.vcf.gz"),
          path("${fix_bam.simpleName}.${interval}.g.vcf.gz.tbi")

    script:
    if (params.gatk_build == 3)
        """
        $gatk -T HaplotypeCaller \
          -R ${refseq} \
          -I ${fix_bam} \
          -L ${interval} \
          -o ${fix_bam.simpleName}.${interval}.g.vcf.gz \
          -ERC GVCF \
          -out_mode EMIT_ALL_SITES
        """
    else if (params.gatk_build == 4)
        """
        $gatk HaplotypeCaller \
          -R ${refseq} \
          -I ${fix_bam} \
          -L ${interval} \
          -O ${fix_bam.simpleName}.${interval}.g.vcf.gz \
          -ERC GVCF \
          -G StandardAnnotation \
          -G AS_StandardAnnotation
        """
}
```

New gatherSampleGVCFs process

```
process gatherSampleGVCFs {

    // Recombine chromosome-level GVCFs into one GVCF per sample

    label 'gatk'
    publishDir "$params.outdir/02_gVCFs", mode: 'copy'
    tag "${sample}"

    input:
    tuple val(sample), path(gvcfs), path(gvcf_indices)
    path refseq
    path "*"

    output:
    tuple path("${sample}.g.vcf.gz"),
          path("${sample}.g.vcf.gz.tbi")
```

```
    script:
    def variantArgs = gvcfs
        .sort { a, b -> a.name <=> b.name }
        .collect { "-V ${it}" }
        .join(' ')

    """
    $gatk CombineGVCFs \
      -R ${refseq} \
      ${variantArgs} \
      -O ${sample}.g.vcf.gz
    """
}
```

The original genotypegVCFs process remains unchanged. It still receives one reconstructed GVCF per sample and performs joint genotyping.

6. Workflow wiring change

The original workflow call to callVariants was replaced with interval channel construction, BAM x interval combination, grouping by sample, and per-sample gathering.

```
hc_intervals = Channel
    .fromPath(
        params.hc_intervals,
        checkIfExists: true
    )
    .splitText()
    .map { interval -> interval.trim() }
    .filter { interval -> interval }

if (params.filter_bams) {

    filterBAMs(
        realignIndels.out,
        params.refseq,
        prepareRef.out
    ) | fixMate

    hc_inputs = fixMate.out.combine(hc_intervals)

    callVariants(
        hc_inputs,
        params.refseq,
        prepareRef.out
    )

} else {

    hc_inputs = realignIndels.out.combine(hc_intervals)

    callVariants(
        hc_inputs,
        params.refseq,
        prepareRef.out
    )
}

sample_gvcfs = callVariants.out
    .map { sample, interval, gvcf, index ->
        tuple(sample, gvcf, index)
    }
    .groupTuple()

gatherSampleGVCFs(
    sample_gvcfs,
    params.refseq,
    prepareRef.out
)

genotypegVCFs(
    gatherSampleGVCFs.out.collect(),
    params.refseq,
    prepareRef.out
)
```

The rest of the downstream workflow remains unchanged: masking, chromosome filtering, phasing, trio splitting, site filtering, region filtering, mutation-rate calculation, and summary statistics.

7. Conda and resource configuration

GATK environment selector

The new gather process must be included in the same GATK Conda selector as callVariants:

```
withName: 'realignIndels|callVariants|gatherSampleGVCFs' {
    if (params.gatk_build == 3) {
```

```
        conda = 'bioconda::gatk=3.8 bioconda::picard=2.23.8'
    } else if (params.gatk_build == 4) {
        conda = 'bioconda::gatk4=4.4.0.0 bioconda::picard=3.1.0 conda-forge::openjdk=17.0.9'
    }
}
```

Process-specific resource settings

```
withName: callVariants {
    cpus = 1
    maxForks = 30
}

withName: gatherSampleGVCFs {
    cpus = 2
    maxForks = 3
}
```

GATK4 HaplotypeCaller was observed to use approximately one CPU per chromosome task, so cpus=1 avoids reserving unused cores.

Executor cap

The effective configuration that worked included an overall local executor cap:

```
executor {
    cpus = 30
}
```

This cap should be tuned to the shared-server context. A lower value leaves more resources for other users. If RepeatModeler should run concurrently with HaplotypeCaller, reduce callVariants maxForks enough to leave approximately 10 executor CPU slots available for RepeatModeler.

8. Cache behavior and rerun strategy

Nextflow cache reuse depends on the task signature, inputs, process definition, configuration, and work directory. The existing work/ directory should be preserved.

• Always use -resume.

• Do not delete work/ between filter experiments or added-offspring runs.

• Changing only downstream filtering should reuse mapping, BAM processing, GenMap, RepeatMasker/RepeatModeler, and GVCFs.

• Adding offspring should reuse existing sample tasks but rerun joint genotyping and downstream trio-dependent steps.

• Changing ratestools.nf, process Conda settings, beforeScript, or relevant parameters may invalidate affected tasks.

• Switching from the cached GitHub workflow to the local modified workflow preserved many upstream tasks because the clone began at the same commit.

Separate output directories

For sensitivity analyses, keep the same work cache but use a new outdir so published results are not overwritten.

```
# Example run-specific config values:
outdir = "results_filters_default"
# Later:
outdir = "results_filters_relaxed"
```

9. Monitoring

Nextflow task status

The interactive Nextflow display reports completion and cache use. A successful scattered run should show 72 callVariants tasks.

Active HaplotypeCaller count

```
pgrep -af HaplotypeCaller | grep -v pgrep | wc -l
```

Inspect active commands

```
ps -C java -o pid,ppid,etime,pcpu,pmem,args |
grep HaplotypeCaller | head -20
```

Monitor GVCF creation

```
find work -name "*.g.vcf.gz"   -printf "%TY-%Tm-%Td %TH:%TM:%TS %12s %p
" |
sort | tail -30
```

Disk monitoring

```
df -h /local
du -sh work
du -sh work/* 2>/dev/null | sort -h | tail
```

Disk monitoring is essential because /local was observed at approximately 97% full during the run.

10. Troubleshooting record

Symptom	Cause	Resolution
Nextflow cannot find acceptable Java	System Java 13	module load java/21.0.1
Mamba root-prefix warning	MAMBA_ROOT_PREFIX mismatch	Set root prefix consistently with active Miniforge installation
configure.sh sed errors	macOS sed syntax and path delimiters	Patch sed usage and manually verify config
Pipeline searches for data.csv / RawData	Defaults remained in config	Replace libraries and readDir with absolute paths
genmap: command not found	Conda rule/path mismatch	Use working GenMap path with beforeScript or corrected PATH
GenMap FASTA does not exist	Reference default remained ref.fa	Use absolute refseq path
BWA cannot open FASTQ	CSV suffix/path mismatch	Match .fq.gz names and correct readDir
Mamba lock unavailable	Orphaned or concurrent mamba processes	Inspect/terminate stale jobs; use a dedicated cache if needed
/dev/null: Permission denied	Incorrect server device permissions	BioHPC admin restored /dev/null to writable character device
Whole-genome HaplotypeCaller takes days	One single-core task per sample	Scatter by chromosome and gather per sample
RepeatModeler appears at 0%	All executor CPU slots occupied by HaplotypeCaller	Reduce callVariants concurrency or wait
GenMapMap reruns	Task signature changed after local workflow/config edits	Expected; stabilize source/config for future cache reuse

11. Final validation before future runs

- Confirm local clone commit and preserve a backup or Git commit of the modified ratestools.nf.
- Run nextflow config and inspect effective values.
- Confirm hc_intervals.txt exists and contains exactly the intended chromosomes.
- Confirm dam and sire exactly match CSV Sample values.
- Confirm chr_file exists or is set to NULL.
- Confirm rm_species is biologically appropriate.
- Confirm FASTQ filenames in CSV exist and use the correct .gz suffix.
- Confirm run_ratestools.sh passes -resume and points to the local ratestools.nf.
- Start inside screen.
- Check active HaplotypeCaller count after launch and reduce executor CPUs if the server is too busy.

12. Version-control recommendation

```
cd /workdir/nina_july2026/silverside_progeny/RatesTools

git status
git add ratestools.nf
git commit -m "Scatter HaplotypeCaller by chromosome and gather sample GVCFs"
git tag ratestools-1.2.5-biohpc-scattered
```

Keep a copy of the analysis-specific nextflow.config and hc_intervals.txt with the project records. The config contains run-specific paths and parent/sample definitions and may not belong in the upstream pipeline repository.

13. Current observed outcome

- The original whole-genome HaplotypeCaller progressed to roughly one-third of the genome only after several days.
- After chromosome scattering, 30 of 72 tasks completed in less than 12 hours in one observed run.
- All alignment, duplicate marking, library merging, and realignment tasks were successfully reused from cache.
- RepeatMasker and RepeatModeler outputs were also recoverable from cache when task signatures matched.

• The modified design uses the server much more effectively and can be scaled by adjusting maxForks and executor CPU limits.

This document records the workflow state reached during the July 2026 debugging and optimization session. Before reusing it on another project, verify paths, sample names, reference assembly, chromosome list, and resource limits.